Category 3

I shall make a UI element for the Dashboard.
So now, ros2 bridge and telemetry will be 2 python files
ros2 bridge will just relay the data sent from Dashboard ui every 2 second (dashboard ui will have one + and - button chumma) and will send all this data to the telemetry logging system which will keep writing into one log.txt
My ros2 bridge links my UI element and my telemetry logging system

For this, my stupid ahh was tryna figure out some way to do scheduling without cron and timers.

The only thing i was returning back to was systemd because logically thats the only guy who can start services with him because he is the first process to ever start in the user space and he is the boss

and i read the task title again

"systemd service orchestration" 🙏🙏🙏🙏🙏🙏🙏🙏 blind

this basically calls for the use of systemctl shit

https://www.exam-labs.com/blog/systemd-unveiled-the-evolution-of-linux-service-management

An introduction to systemd and its role in the boot process. - GeeksforGeeks

systemd is the first processes run by the kernel. it has the power to manage who runs what and when.

Systemd Units: Everything in systemd is a "unit." Each unit is defined by a configuration file.

- .service: A standard background process
- .mount: A file system mount point
- .timer: A scheduled job
- .target: A group of units

managing services: systemctl

analysis of logs: journalctl

We will be making use of networkTarget

So as planned, i made a ui

UI and the whole python part: i was lazy and time illa, so i asked copilot to cook them files.

i have reviewed and added code explanation comments

bro i first asked copilot to make two proper programs which use IPC and communicate ra

bro pulledup with function calling from both `telemetry_logger.py` and `ros2_bridge.py`

🙏🙏🙏🙏🙏🙏🙏🙏🙏

I asked it to implement it with the help of sockets concept

so the ui will keep sending data on port 5000

ros2 bridge will listen on 5000 and relay it to 5001

and telemetry will take from 5001 and put in logs.txt

the data format exhanged between is json

also the reason i chose sockets concept instead of some shared file was cuz like shared files felt too much of read and write and some race condition anxiety if more than one instances are created

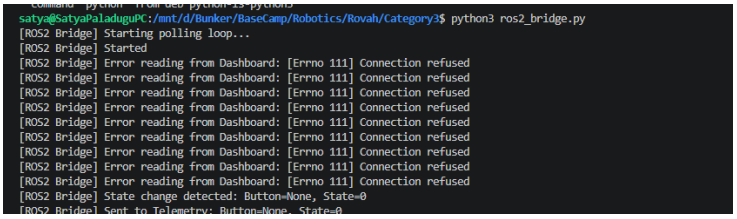
sockets means every few seconds we can keep polling

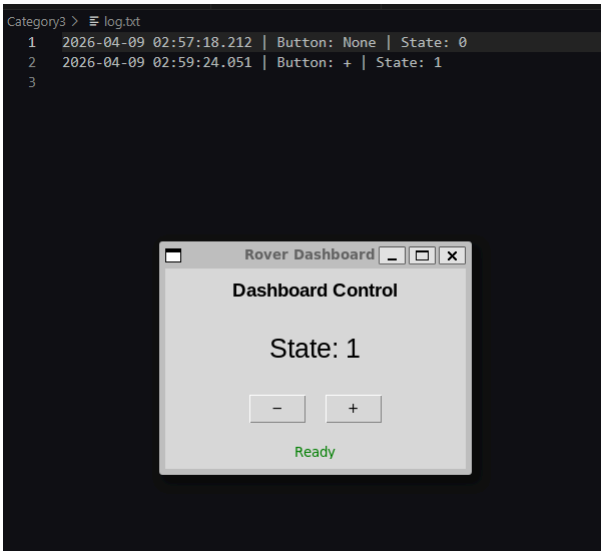
this is all i could think of

so here is the whole thing without the whole systemd service orchestration

Telemetry

ros2_bridge





making services

lil bit i wanted to do the code service writing part myself for the satisfaction

Creating a Service in Linux: A Step-by-Step Guide | by The_Anshuman | Medium

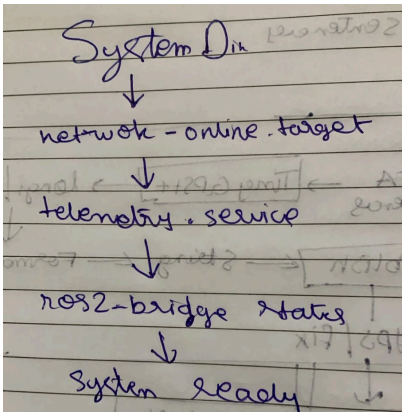
first let's see the folder system

```
root@SatyaPaladuguPC:~# ls /etc/systemd/system
cloud-config.target.wants                  dbus-org.freedesktop.resolve1.service   sockets.target.wants
cloud-final.service.wants                  dbus-org.freedesktop.timesync1.service  sudo.service
cloud-init.target.wants                    final.target.wants                      sysinit.target.wants
dbus-fi.w1.wpa_supplicant1.service         getty.target.wants                      syslog.service
dbus-org.freedesktop.Avahi.service         multi-user.target.wants                 timers.target.wants
dbus-org.freedesktop.ModemManager1.service paths.target.wants
root@SatyaPaladuguPC:~#
```

NetworkTarget

First lets discuss about our design

technically, one should go with this:



idk why chatgpt was suggesting me to go with some group method. seemed over complicated

- The services can start as standalone programs, but **extra credits** 😳 **will be awarded** for configuring them to autostart together.
- Dependency ordering must be ensured — services should start only when their dependencies are satisfied. 💦

```
[Unit]
Description=Telemetry Logging Service
After=network-online.target
Wants=network-online.target
[Service]
ExecStart=/usr/bin/python3 telemetry.py
Restart=always
RestartSec=2
[Install]
WantedBy=multi-user.target
```

This is for telemetry.service.

it will start only after network is online, it needs network to be online

```
[Unit]
Description=ROS2 Bridge Service
After=network-online.target telemetry.service
Wants=network-online.target
Requires=telemetry.service

[Service]
ExecStart=/usr/bin/python3 /path/to/ros2_bridge.py
Restart=always
RestartSec=2
```

```
[Install]
WantedBy=multi-user.target
```

This is for ROS2 Bridge Service

it will start only after netowk is online and Telemetry is active

`Wants=network-online.target` → "Id like network, but I'll try anyway"

`Requires=telemetry.service` → "If telemetry is dead, KILL ME TOO"

BONUS PART

Now let's talk about autostart.

we have set "wantedBy = multi-user.target"

multiuser target is the default one for services

so from what i understand, each service has to start one by one

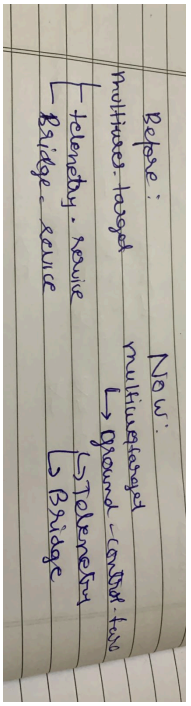
sudo systemctl enable telemetry

sudo system enabl bridge

this doesnt make it start together and it makes it standalone

SO

I shouldi introduce one target called
ground_control.target which waits till the network is up
And then the other 2 services are required by
ground_control
so i do not need to do systemctl enable telemetry or
ros_bridge separately, instead enabling
ground_control target would simply make sure the
other two run regardless



sudo systemctl enable ground-control.target
sudo systemctl start ground-control.target

bro but its more like simply putting them under the same house and then starting

or like its simply just grouping them

(NEVER MIND, CHATGPT WAS TRYNA HELP 😭)

but anywasy

So,

```
[Unit]
Description=Telemetry Logging Service
After=network-online.target
Wants=network-online.target
[Service]
ExecStart=/usr/bin/python3 /mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/telemetry.py
Restart=always
RestartSec=2
[Install]
WantedBy=ground-control.target
```

```
[Unit]
Description=ROS2 Bridge Service
After=network-online.target telemetry.service
Wants=network-online.target
Requires=telemetry.service

[Service]
ExecStart=/usr/bin/python3 /mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/ros2_bridge.
py
Restart=always
RestartSec=2

[Install]
WantedBy=ground-control.target
```

```
[Unit]
Description=Rover Ground Control System
After=network-online.target
Wants=network-online.target

[Install]
WantedBy=multi-user.target
```

Where files sit

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo cp ground-control.target /etc/systemd/system/
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo cp ros2-bridge.service /etc/systemd/system/
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo cp telemetry.service /etc/systemd/system/
```

put everything at

`/etc/systemd/system/telemetry.service`
`/etc/systemd/system/ros2-bridge.service`
`/etc/systemd/system/ground-control.target`

OK NOW LETS ENABLE

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo systemctl daemon-reload
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo systemctl enable ground-control.target
Created symlink /etc/systemd/system/multi-user.target.wants/ground-control.target → /etc/systemd/system/ground-control.target.
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ sudo systemctl start ground-control.target
```

oh and btw

wsl2 doesn't support systemd

so you have to go to sudo nano /etc/wsl.conf and set systemd = true

I AM TRYING TO RUN THE UI DASHBOARD VIA THE ROS2_BRIDGE.PY

(its messing up everything)

NOW LET's TALK BREAKING

There are multiple ways we can break

1. Some chappar changes location of the files
2. .service is altered
3. Someone else is already using the port (mine)

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ journalctl -u telemetry.service -f
Apr 09 04:28:56 SatyaPaladuguPC systemd[1]: Started Telemetry Logging Service.
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]: Exception in thread Thread-1 (socket_listen):
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]: Traceback (most recent call last):
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:   File "/usr/lib/python3.10/threading.py", line 1016, in _bootstrap_inner
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:     self.run()
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:   File "/usr/lib/python3.10/threading.py", line 953, in run
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:     self._target(*self._args, **self._kwargs)
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:   File "/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/telemetry_logger.py", line 32, in socket_listen
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]:     self.socket_server.bind(("localhost", 5001))
Apr 09 04:28:56 SatyaPaladuguPC python3[2663]: OSError: [Errno 98] Address already in use
```

funny story

i showed you earlier without systemd right

i forgot to kill the process 😭😭😭😭😭😭😭😭😭😭😭

eitherways, we broke it for the demo

THIS REMINDS ME

4. if there was a shared file, and many people wanna write at once, they wont be able to access only
5. the shared files can also loose permissions
4. Dependencies and environment issue or bad code (again mine)

your bridge service was trying to launch the Tkinter dashboard from systemd, and systemd is running headless in WSL, so Tk crashed with no DISPLAY.

i asked ai to explain the error cuz idk much bout the beef between wsl and systemd

but long story short, its related to headless mode

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ sudo systemctl status ros2-bridge.service
● ros2-bridge.service - ROS2 Bridge Service
     Loaded: loaded (/etc/systemd/system/ros2-bridge.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-04-09 04:47:28 IST; 27s ago
   Main PID: 628 (python3)
      Tasks: 3 (limit: 9360)
     Memory: 12.2M
        CPU: 136ms
     CGroup: /system.slice/ros2-bridge.service
             └─628 /usr/bin/python3 /mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/ros2_bridge.py

Apr 09 04:47:28 SatyaPaladuguPC systemd[1]: Started ROS2 Bridge Service.
Apr 09 04:47:28 SatyaPaladuguPC python3[629]: Traceback (most recent call last):
Apr 09 04:47:28 SatyaPaladuguPC python3[629]:   File "/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/dashboard_ui.py", line 113, in <module>
Apr 09 04:47:28 SatyaPaladuguPC python3[629]:     root = tk.Tk()
Apr 09 04:47:28 SatyaPaladuguPC python3[629]:   File "/usr/lib/python3.10/tkinter/__init__.py", line 2299, in __init__
Apr 09 04:47:28 SatyaPaladuguPC python3[629]:     self.tk = _tkinter.create(screenName, baseName, className, interactive, wantobjects, useTk, sync, use)
Apr 09 04:47:28 SatyaPaladuguPC python3[629]: _tkinter.TclError: no display name and no $DISPLAY environment variable
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$
```

5. connection refusal (continuation of previous one)

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah$ sudo journalctl -u ros2-bridge -f
Apr 09 04:52:13 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:15 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:17 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:19 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:21 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:23 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:25 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:27 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:29 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
Apr 09 04:52:31 SatyaPaladuguPC python3[789]: [ROS2 Bridge] Error reading from Dashboard: [Errno 111] Connection refused
```

6. Permission change

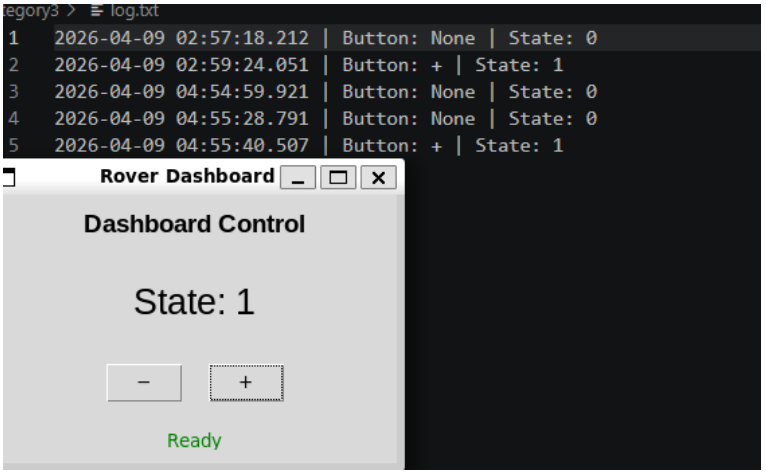
some anti nationalist comes and changes telemetry.py to chmod 000

i cannot think of any other methods

best fix:

```
satya@SatyaPaladuguPC:/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3$ python3 dashboard_ui.py
[Dashboard] Socket server started on port 5000
[Dashboard] Client connected: ('127.0.0.1', 57602)
[Dashboard] Client connected: ('127.0.0.1', 54384)
```

run the ui on another window



voila

but we need all this to run together right

only option now is add dashboard.py also to services

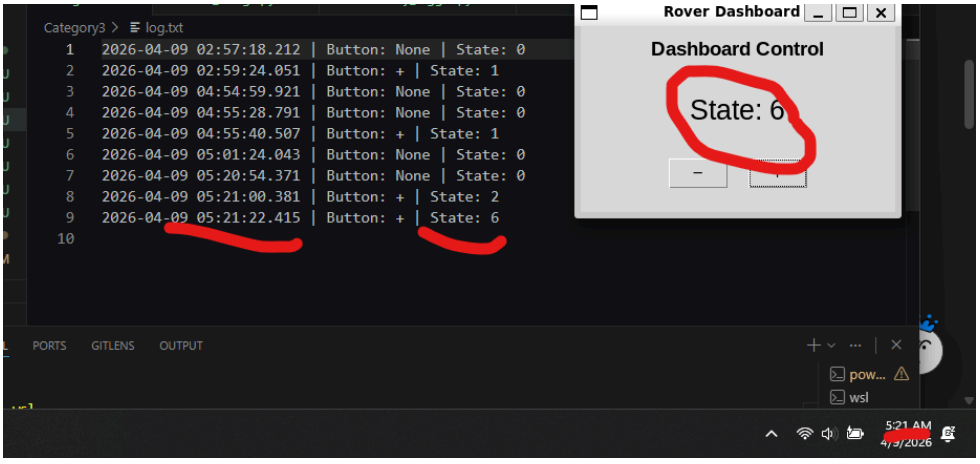
```
[Unit]
Description=Dashboard UI Service (Socket Server)
After=network-online.target
Wants=network-online.target

[Service]
ExecStart=/usr/bin/python3 /mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3/dashboard_ui.py
WorkingDirectory=/mnt/d/Bunker/BaseCamp/Robotics/Rovah/Category3
Environment=DASHBOARD_HEADLESS=1
Environment=PYTHONUNBUFFERED=1
Restart=always
RestartSec=2

[Install]
WantedBy=ground-control.target
```

but its not working and i am out of time so i am ditching it👍

done ra 🙏



ChatGPT chat link: https://chatgpt.com/share/69d6d130-0390-8321-ac46-bad449e1fd08

Code explanation

```
"""
ROS2 Bridge - Polls dashboard state every 2 seconds via socket
Relays to telemetry logger via socket
"""
import json
import os
import socket
import sys
import threading
import time
from datetime import datetime
from pathlib import Path


DASHBOARD_HOST = "localhost"
DASHBOARD_PORT = 5000
TELEMETRY_HOST = "localhost"
TELEMETRY_PORT = 5001
POLL_SECONDS = 2


class ROS2Bridge:
    def __init__(self):
        self.running = False
        self.last_state = None
        self.last_button = None

    def _can_connect(self, host, port, timeout=0.5):
```

```
        """Return True if a TCP endpoint is reachable."""
        try:
            with socket.create_connection((host, port), timeout=timeout):
                return True
        except OSError:
            return False

    def read_dashboard_state(self):
        """Connect to dashboard socket and get current UI state."""
        try:
            with socket.create_connection((DASHBOARD_HOST, DASHBOARD_PORT), timeout=1.
0) as sock:
                data = sock.recv(2048).decode("utf-8")

            state_data = json.loads(data)
            return state_data.get("state"), state_data.get("button")
        except Exception as exc:
            return None, None

    def send_to_telemetry(self, button, state):
        """Send log data to telemetry logger via socket."""
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S.%f")[:-3]
        payload = {
            "timestamp": timestamp,
            "button": button,
            "state": state,
        }

        try:
            with socket.create_connection((TELEMETRY_HOST, TELEMETRY_PORT), timeout=1.
0) as sock:
                sock.sendall(json.dumps(payload).encode("utf-8"))
            print(f"[ROS2 Bridge] Sent to Telemetry: Button={button}, State={state}")
        except Exception as exc:
            print(f"[ROS2 Bridge] Error sending to Telemetry: {exc}")

    def poll_and_relay(self):
        """Poll dashboard every 2 seconds and relay changes."""
        print("[ROS2 Bridge] Starting polling loop...")
        consecutive_failures = 0
        max_failures_before_warn = 3

        while self.running:
            state, button = self.read_dashboard_state()

            if state is not None:
                consecutive_failures = 0
                if state != self.last_state or button != self.last_button:
                    print(f"[ROS2 Bridge] State change detected: Button={button}, State
={state}")
                    self.send_to_telemetry(button, state)
                    self.last_state = state
                    self.last_button = button
            else:
                consecutive_failures += 1
                if consecutive_failures == max_failures_before_warn:
```

```
                    print(
                        "[ROS2 Bridge] Dashboard unreachable. "
                        "Check if dashboard.service is running: sudo systemctl status d
ashboard.service"
                    )

            time.sleep(POLL_SECONDS)

    def start(self):
        """Start bridge polling."""
        self.running = True
        print("[ROS2 Bridge] Started")
        self.poll_and_relay()

    def stop(self):
        """Stop bridge."""
        self.running = False
        print("[ROS2 Bridge] Stopped")


if __name__ == "__main__":
    bridge = ROS2Bridge()

    try:
        bridge.start()
    except KeyboardInterrupt:
        print("\n[ROS2 Bridge] Shutting down...")
        bridge.stop()
```

Reading the Dashboard (read_dashboard_state)

Every 2 seconds, the Bridge acts as a **Client**.

It tries to open a TCP connection to `localhost:5000` (where the Dashboard UI is acting as a **Server**).

If it connects, it receives a tiny JSON string (e.g., {"state": 5, "button": "+"}), parses it into a Python dictionary, and returns the values. If the connection fails, it returns `None, None`.

Processing and Filtering (poll_and_relay)

This is the infinite main loop while self.running:.

It gets the data from step 2.

It checks: if state != self.last_state or button != self.last_button:

Meaning: *Did they click the button since I last checked?*

If yes, it triggers step 4 and updates its memory of the last_state.

If no, it just does time.sleep(2) and circles back.

For edge cases, functions like *can*connect are there

Sending to Telemetry (send_to_telemetry)

When a change is detected, the Bridge again acts as a Client.

It packages the new data, adds a precise timestamp, and opens a TCP connection to `localhost:5001` (where the Telemetry Logger is acting as a **Server**).

It sends the JSON payload: {"timestamp": "...", "button": "+", "state": 5} and immediately closes the connection.

```
"""
Telemetry Logger - Listens on socket (port 5001) for log requests
Writes to log.txt
"""
import json
import socket
from pathlib import Path


class TelemetryLogger:
    def __init__(self):
        self.log_file = Path(__file__).parent / "log.txt"
        self.running = False
        self.socket_server = None
        self.processed_count = 0

    def write_log(self, timestamp, button, state):
        """Write single entry to log.txt"""
        with open(self.log_file, 'a') as f:
            log_line = f"{timestamp} | Button: {button} | State: {state}\n"
            f.write(log_line)
            print(f"[Telemetry] Logged: {log_line.strip()}")

        self.processed_count += 1

    def socket_listen(self):
        """Listen for incoming log requests on port 5001"""
        self.socket_server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.socket_server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.socket_server.bind(("localhost", 5001))
        self.socket_server.listen(5)

        print("[Telemetry] Listening on port 5001...")

        while self.running:
            try:
                self.socket_server.settimeout(1)
                client, addr = self.socket_server.accept()

                # Receive log data
                data = client.recv(1024).decode('utf-8')
                client.close()

                if data:
                    try:
                        log_data = json.loads(data)
                        self.write_log(
                            log_data.get("timestamp"),
                            log_data.get("button"),
                            log_data.get("state")
                        )
                    except json.JSONDecodeError:
                        print("[Telemetry] Invalid JSON received")
            except socket.timeout:
```

```
                continue
            except:
                break

    def start(self):
        """Start the logger listening on socket"""
        self.running = True
        print("[Telemetry] Started")
        self.socket_listen()

    def stop(self):
        """Stop the logger"""
        self.running = False
        if self.socket_server:
            self.socket_server.close()
        print(f"[Telemetry] Stopped (processed {self.processed_count} events)")


if __name__ == "__main__":
    logger = TelemetryLogger()

    try:
        logger.start()
    except KeyboardInterrupt:
        print("\n[Telemetry] Shutting down...")
        logger.stop()
```

.services

https://linuxize.com/post/how-to-create-a-systemd-service/#:~:text=Where%20should%20I%20put%20my,use%20sudo%20systemctl%20restart%20myservice%20.

bro explained everything in this page

[Unit] Section

The `[Unit]` section contains metadata and dependency declarations.

Common directives:

- `Description=` — a short human-readable name shown in `systemctl status` output.
- `After=` — start this service only after the listed units are active. Does not create a dependency; use `Requires=` for hard dependencies.
- `Requires=` — this service requires the listed units. If they fail, this service fails too.
- `Wants=` — like `Requires=`, but the service still starts even if the listed units fail.
- `Documentation=` — a URL or man page reference for the service.

[Service] Section

The `[Service]` section defines the process and how systemd manages it.

Common directives:

- `Type=` — the startup type. See Service Types below.
- `ExecStart=` — the command to run when the service starts. Must be an absolute path.
- `ExecStop=` — the command to run when the service stops. If omitted, systemd sends `SIGTERM`.

- `ExecReload=` — the command to reload the service configuration without restarting.
- `Restart=` — when to restart automatically. See Restart Policies below.
- `RestartSec=` — how many seconds to wait before restarting. Default is 100ms.
- `User=` — run the process as this user instead of root.
- `Group=` — run the process as this group.
- `WorkingDirectory=` — set the working directory for the process.
- `Environment=` — set environment variables: `Environment="KEY=value"`.
- `EnvironmentFile=` — read environment variables from a file.

[Install] Section

The `[Install]` section controls how the service is enabled.

- `WantedBy=multi-user.target` — the most common value. Enables the service for normal multi-user (non-graphical) boot.
- `WantedBy=graphical.target` — use this if the service requires a graphical environment.